

Neural Networks for Multiclass Classification

To adapt a neural network for a multiclass classification problem (e.g., classifying digits 0–9), we modify the **output layer**. Instead of using a single logistic (sigmoid) unit, we use the **softmax regression** model as the final layer.

The Softmax Output Layer

Architecture

The architecture of the hidden layers remains the same (e.g., using ReLU activations). The change is in the output layer:

- **Binary Classification:** The output layer has **1 unit** (neuron) with a **sigmoid** activation.
- **Multiclass Classification:** The output layer has **N units** (neurons), one for each class. For digit recognition (0–9), this means the output layer will have **10 units**. This is called a **softmax layer**.

Forward Propagation

1. **Hidden Layers:** The activations for the hidden layers ($\vec{a}^{[1]}$, $\vec{a}^{[2]}$, etc.) are computed exactly as before (e.g., using ReLU).
2. **Output Layer ($\vec{a}^{[3]}$):**

- **Step 2a (Compute Z):** The final layer first computes a linear score, z_j , for each of the 10 output units. The input is the activation vector from the previous layer, $\vec{a}^{[2]}$.

$$z_j^{[3]} = \vec{w}_j^{[3]} \cdot \vec{a}^{[2]} + b_j^{[3]} \quad (\text{for } j = 1 \text{ to } 10)$$

- **Step 2b (Compute A):** The **softmax function** is then applied *across all 10 z values* to convert these scores into probabilities.

$$a_j^{[3]} = P(y = j | \vec{x}) = \frac{e^{z_j^{[3]}}}{\sum_{k=1}^{10} e^{z_k^{[3]}}}$$

The result, $\vec{a}^{[3]}$, is a vector of 10 probabilities (one for each class) that will sum to 1.

A Key Property of Softmax

Unlike the sigmoid, ReLU, and linear activation functions, the softmax activation is **not element-wise**.

- For sigmoid/ReLU, the output of a single neuron (a_j) depends *only* on its corresponding linear input (z_j).
- For **softmax**, the output of a single neuron (a_j) depends on **all** the linear inputs (

$z_1, z_2, \dots, z_{10}$) because of the sum in the denominator.

TensorFlow Implementation

The 3-step process in TensorFlow is updated as follows:

1. Define the Model

The output layer is changed to a Dense layer with N units and the 'softmax' activation.

```
1 model = Sequential([
2     Dense(units=25, activation='relu'),
3     Dense(units=15, activation='relu'),
4     # Output layer has 10 units and softmax activation
5     Dense(units=10, activation='softmax')
6 ])
```

2. Compile the Model

We must use a new loss function designed for multiclass classification:

SparseCategoricalCrossentropy.

```
1 model.compile(loss='sparse_categorical_crossentropy')
```

- **CategoricalCrossentropy:** This is the TensorFlow name for the softmax loss function ($L = -\log(a_j)$) that we saw previously.
- **Sparse:** This refers to the format of your true labels, y . It means your y labels are **single numbers** (e.g., 0, 1, 7, 9). This is in contrast to a "one-hot" format.

3. Train the Model

This step remains exactly the same as before.

```
1 model.fit(X, Y, epochs=...)
```

Important Warning

Do not use the code exactly as written above.

While this code *will* work, it is **not** the recommended or most numerically stable version. A "better" version, which is more accurate, will be shown in the next video.